Experiment 03: Continuous Integration Pipeline using Jenkins / GitHub Actions

Experiment No.	03
Mapped CO	CO3
Aim	To set up a Continuous Integration (CI) pipeline that automatically builds and tests an application on every code change using GitHub Actions, with Jenkins as an alternative.
Tools / Software	GitHub account; Jenkins (optional); A sample application (Python or Node.js); Web browser / terminal

Theory

Continuous Integration (CI) is the practice of frequently merging developer changes into a shared mainline repository. Each change is automatically verified through a build and test process so that integration errors and defects are detected early. A CI server or cloud CI service checks out the latest source code, installs dependencies, builds the application and runs test cases. The final result is shown as pass or fail feedback to the development team.

GitHub Actions implements CI as code using YAML workflow files stored in the .github/workflows directory of the repository. A workflow is triggered by events such as push or pull_request and is made up of jobs and steps that run on a hosted runner. Jenkins can perform the same task through a Pipeline job using a Jenkinsfile and webhooks from GitHub.

Procedure

1. Create or open a GitHub repository containing a sample Python or Node.js application with a test suite.
2. Create the folder .github/workflows in the repository and add a workflow file named ci.yml.
3. Define workflow triggers for push and pull_request events on the main branch.
4. Add workflow steps to check out the code, set up the runtime, install dependencies and execute tests.
5. Commit and push the workflow file to GitHub.
6. Open the Actions tab in GitHub and observe the workflow run. Confirm that the build and test jobs complete successfully with a green check.
7. As an alternative, create a Jenkins Pipeline job, point it to the repository, configure the GitHub webhook, and use a Jenkinsfile containing build and test stages.

Sample Code / Commands

.github/workflows/ci.yml (Python example)

```
name: CI

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
```

```
  build-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.12'
      - run: pip install -r requirements.txt
      - run: pytest -q
```

Jenkinsfile (declarative pipeline alternative)

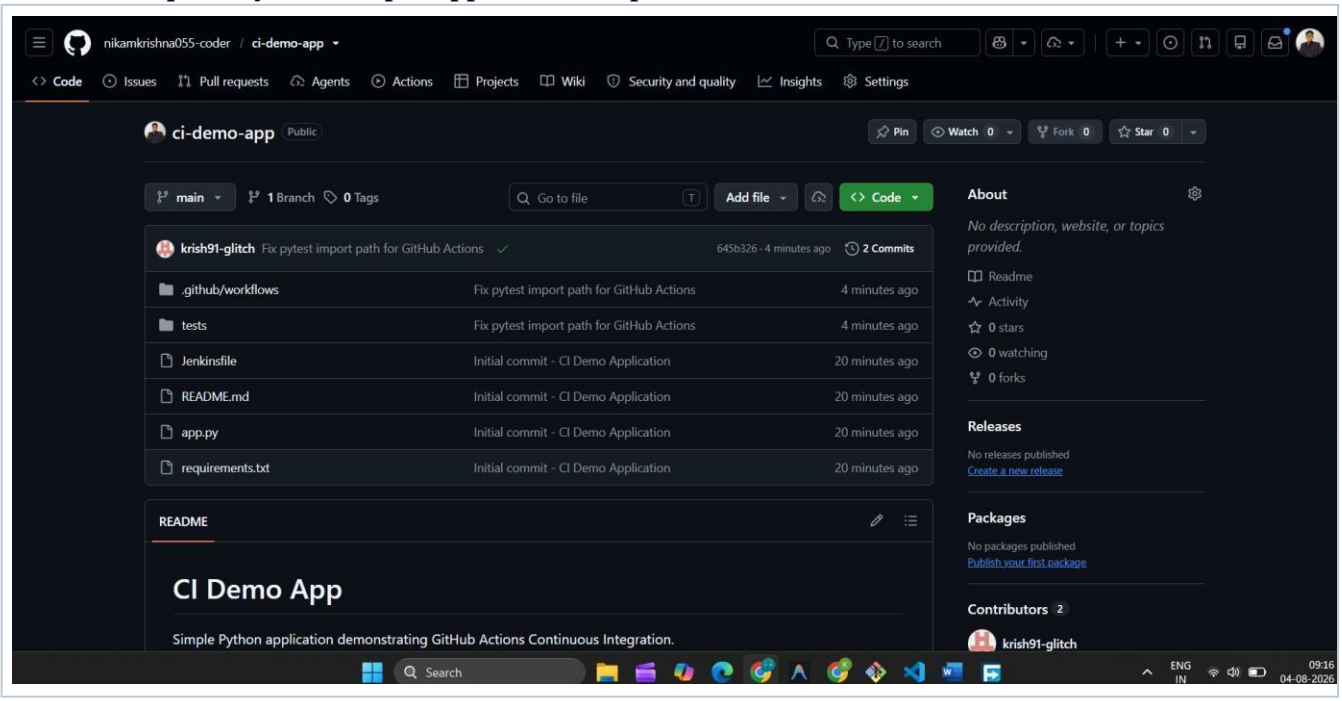
```
pipeline {
  agent any
  stages {
    stage('Build') {
      steps { sh 'pip install -r requirements.txt' }
    }
    stage('Test') {
      steps { sh 'pytest -q' }
    }
  }
}
```

Observation / Experiment Output

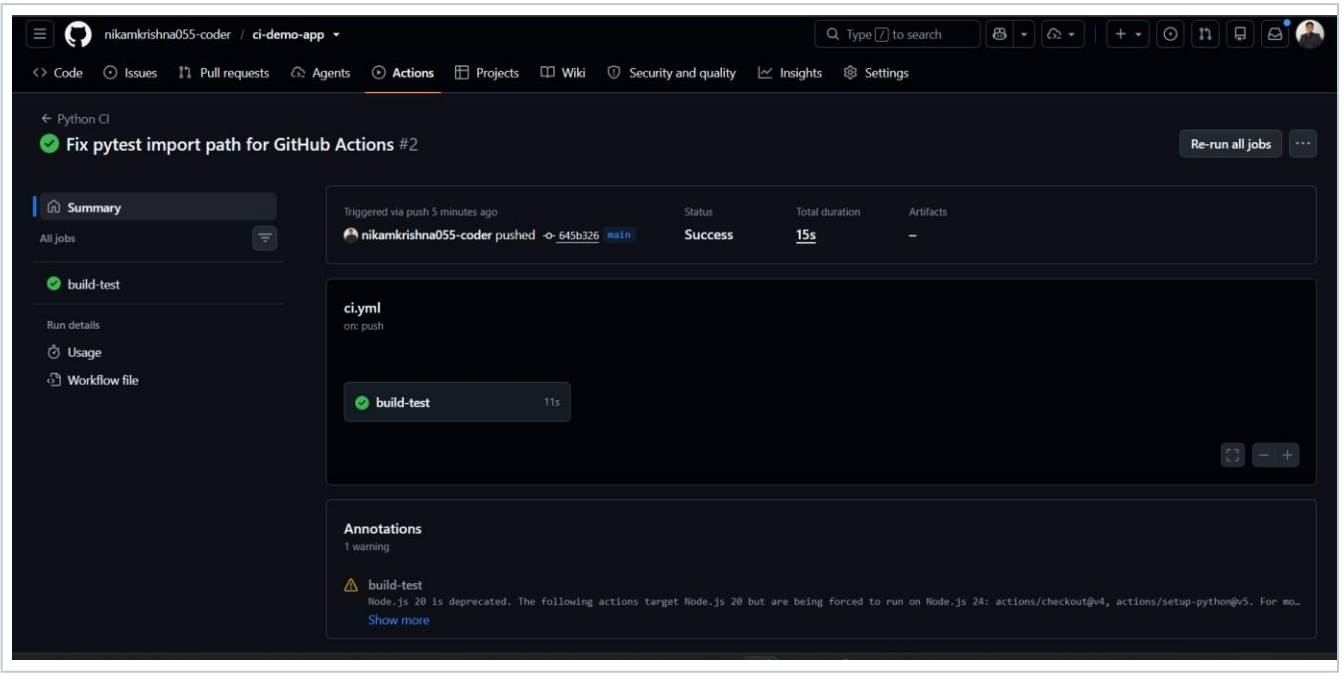
Instructions to students: Paste screenshots or copied outputs in the boxes below after completing CI workflow setup and execution. Keep screenshots clear and readable.

Name	Roll No.	Batch	Date
Sagar Pradhan	49	B2	07-08-2026

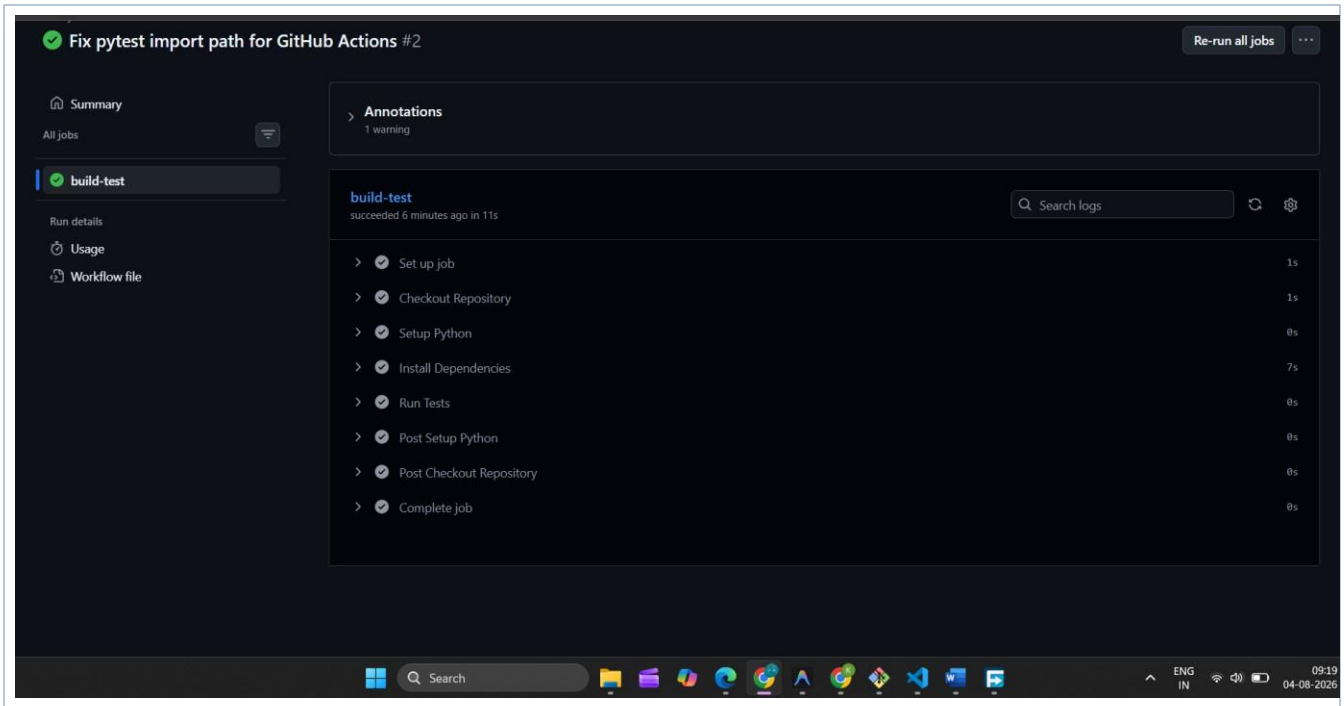
1. GitHub Repository and Sample Application Output



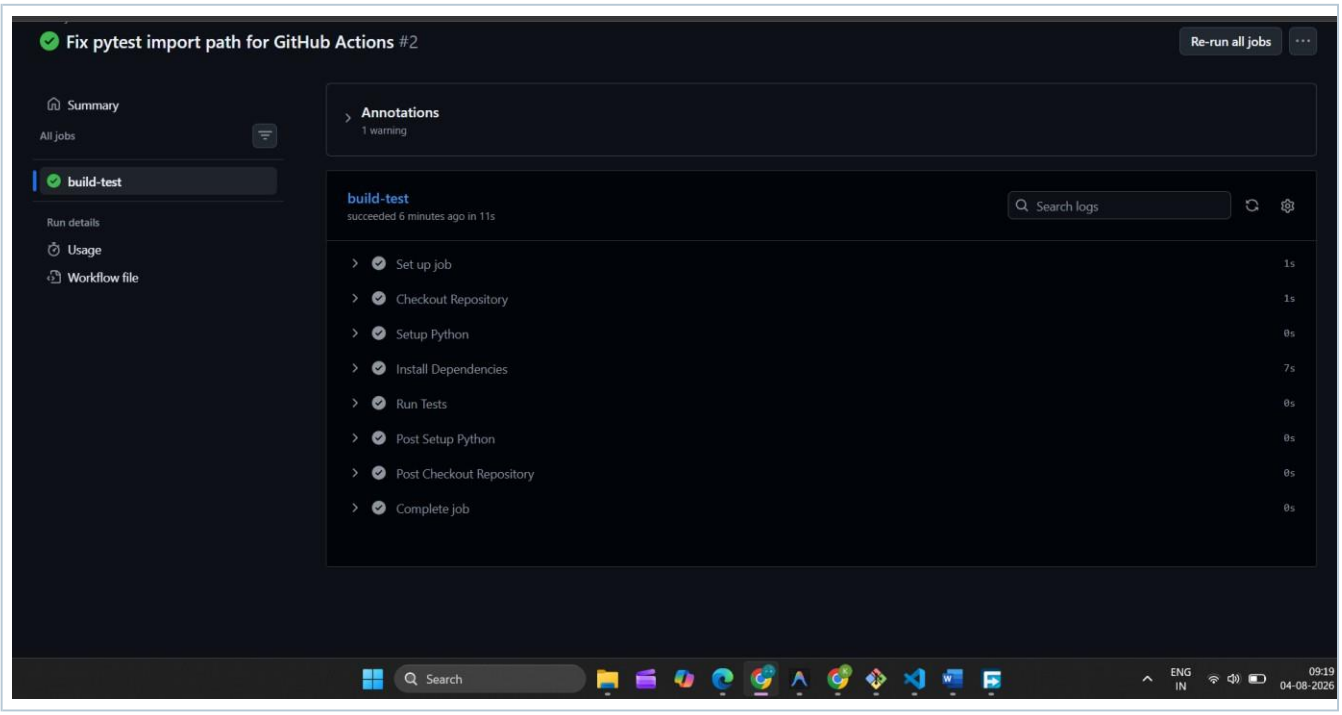
2. CI Workflow File / Jenkinsfile Output



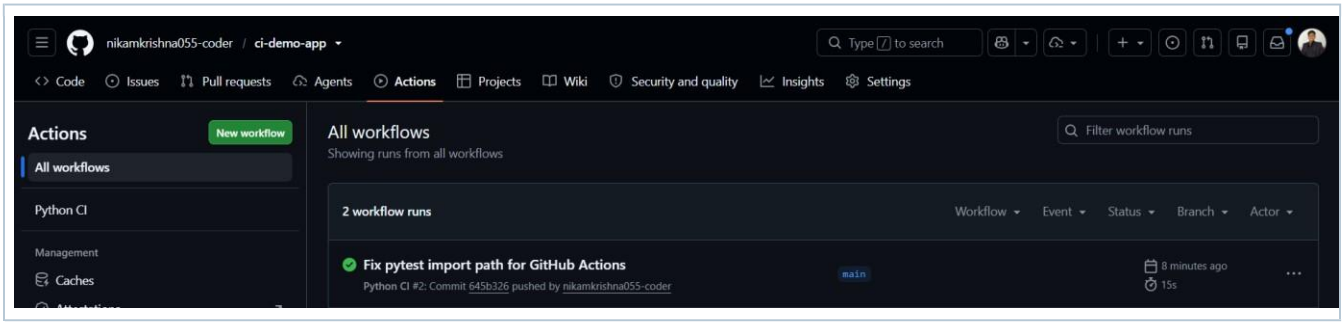
3. Pipeline Trigger Output



4. Build and Test Execution Output



5. Successful CI Status Output



GitHub Repository Link:

_____

GitHub Actions / Jenkins Build Link:

_____

Final Observation Statement:

Write 2-3 lines summarizing the result of the experiment.

Expected Output

On every push or pull request, the CI workflow should run automatically, install dependencies, execute the test suite and report a pass/fail status in the GitHub Actions tab or Jenkins build history.

Conclusion

An automated Continuous Integration pipeline was configured so that each committed code change triggers a build and test cycle, providing rapid feedback on code quality and application health.

Viva / Review Questions

1. What problem does Continuous Integration solve?
2. What is the role of a webhook in a CI pipeline?
3. Differentiate Continuous Integration, Continuous Delivery and Continuous Deployment.
4. What is a runner in GitHub Actions?

Marks: ____/ 10 Date: _____ Signature of Faculty: _____